Windows 환경에서 MCP로 구성하는 AI 기반 모바일 앱 보안 감사

확장판 · MCP · Tool · Skill · Audit System · Guardrails

Windows 기반 모바일 앱 보안 감사 환경을 MCP와 Skill 중심으로 어떻게 설계할지 단계별로 설명합니다.

작성일: 2026-04-24 · 범위: Android 앱 감사, MCP Server, Skill 설계, 운영 통제

1. 발표 목표와 범위

발표 목표

- MCP 기반 도구 연결 방식을 이해한다
- Tool, MCP Server, Skill을 구분한다
- Windows 기반 모바일 앱 감사 환경을 설계한다
- 각 도구의 적용 위치와 연결 방식을 정리한다
- AI 기반 감사 시스템의 운영 원칙을 설명한다

다루는 영역

- Android 앱 정적 분석
- 런타임 동적 분석
- 네이티브 코드 분석
- WebView와 네트워크 분석
- 증거 수집과 보고서화
- 운영 통제와 승인 게이트

시작점

핵심은 도구를 많이 붙이는 것이 아니라,
도구 결과를 하나의 감사 절차와 증거 체계로 묶는 것이다.

2. 모바일 앱 감사의 작업 구조

모바일 앱 감사는 복합 작업이다

1. 감사 범위와 분석 대상을 확정한다.
2. APK, IPA, 소스 코드와 기본 메타데이터를 확보한다.
3. Manifest, 권한, 컴포넌트, 코드 패턴을 정적으로 살핀다.
4. 디바이스에서 실행해 화면, 로그, 함수 흐름을 수집한다.
5. `.so`, WebView, API 트래픽까지 연결해 본다.
6. 마지막에 증거를 정리하고 보고서 초안을 만든다.

왜 어렵나

- 도구가 많고 명령어가 흩어져 있다
- 정적, 동적, 네트워크 결과가 따로 논다
- 증거 저장 규칙이 없으면 재검증이 어렵다
- 보고서 품질이 감사자 경험에 크게 의존한다

AI Agent를 붙이는 이유는 명령어를 대신 입력시키는 데 있지 않고, **도구 간 컨텍스트를 이어 주고 증거를 보고서로 연결하는 데** 있다.

3. AI Agent란 무엇인가

일반 LLM

- 설명, 요약, 코드 생성에 강함
- 외부 도구 실행 능력은 없음
- 파일, 디바이스, 프록시를 직접 조작하지 못함

AI Agent

- 목표를 작업 단위로 분해
- 필요한 Tool을 선택
- 실행 결과를 읽고 다음 행동 결정
- 최종 결과를 Finding, 표, 보고서로 정리

보안 감사에서의 Agent 역할

- 자연어 요청을 ADB, Frida, Ghidra 작업으로 변환
- 로그와 코드에서 이상 징후를 요약
- 증거를 Finding 후보에 연결
- MASVS 기준으로 보고서 초안 작성

4. Tool Calling의 동작 방식

기본 흐름

1. Tool 정의 등록
2. 사용자 요청 해석
3. 실행할 Tool 선택
4. 파라미터 생성
5. Tool 실행
6. 실행 결과 수신
7. 결과를 보고서나 요약으로 통합

보안 감사 예시

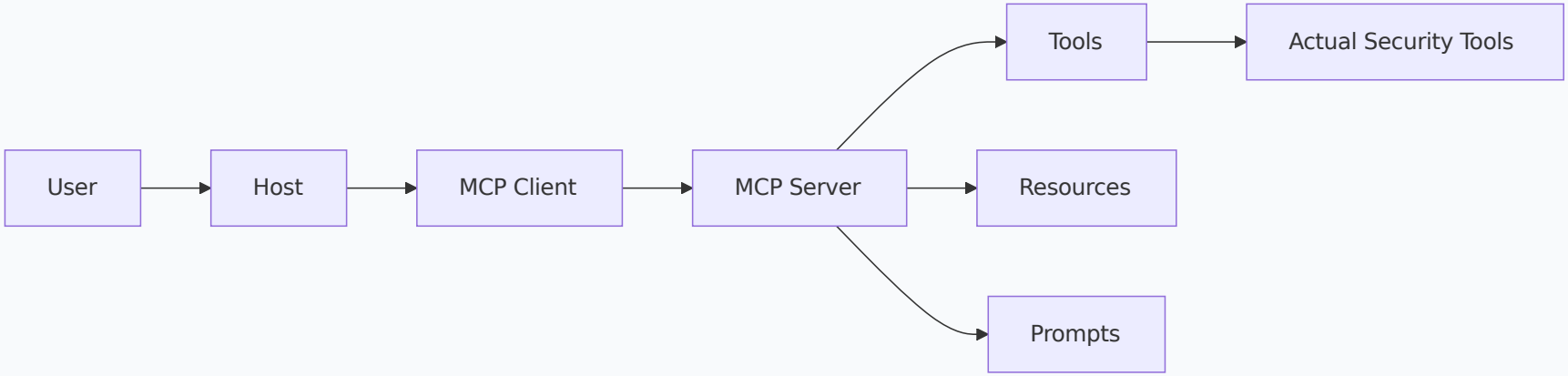
```
{
  "tool": "get_device_info",
  "arguments": {
    "device_id": "R5CR123ABC4"
  }
}
```

- 사용자가 디바이스 상태를 묻는다
- Agent가 적절한 Tool을 고른다
- 런타임이 실행한다
- 결과를 앱 버전, 권한, 화면 상태로 정리한다

중요한 점은 모델이 모든 것을 할 수 있는 것이 아니라, **등록된 Tool 범위 안에서만 움직인다는 것**이다.

5. MCP의 구조

MCP 아키텍처


User
Host
MCP Client
MCP Server
Tools
Actual Security Tools
Resources
Prompts

MCP 구성 요소

- Host: Claude Desktop, Claude Code, Codex CLI, Cursor 등
- MCP Client: Host 안에서 Server와 통신
- MCP Server: 도구 기능을 표준 인터페이스로 노출
- Tool: 실행 함수
- Resource: 참고 데이터
- Prompt: 반복 작업용 템플릿

MCP는 AI용 연결 규격이다. 여러 도구를 일관된 방식으로 붙이게 해준다.

6. MCP, Tool, Skill을 섞지 않기

구분	의미	예시
Tool	AI가 호출하는 단일 실행 기능	`get_screenshot`, `run_script`
MCP Server	여러 Tool을 묶은 서버	Android MCP, Burp MCP
MCP	Client와 Server가 통신하는 규격	JSON-RPC 기반 프로토콜
Skill	절차와 판단 기준을 담은 플레이북	Mobile Audit Skill

용어를 이렇게 쓰는 편이 정확하다

- 부정확: MCP로 스크린샷을 찍는다
- 정확: Android MCP Server의 스크린샷 Tool을 호출한다
- 부정확: MCP가 보고서를 쓴다
- 정확: Agent가 Skill 기준으로 여러 Tool 결과를 정리한다

7. Skill은 무엇을 담당하는가

Skill의 역할

- 반복 감사 절차 표준화
- 도구 호출 순서 정의
- 판단 기준 제공
- 보고서 템플릿 제공
- 증거 저장 규칙 정의

예시 구조

```
mobile-app-security-audit/
  SKILL.md
  references/
  templates/
  scripts/
```

- 입력값
- 사용 가능한 Tool 목록
- 승인 필요 작업
- 증거 수집 위치
- Finding 작성 형식

Skill은 도구가 아니라 **감사자의 노하우를 문서화한 실행 규칙**이다.

8. 기본 도구와 MCP 도구

일반 CLI Agent의 기본 도구

- 파일 읽기
- 파일 검색
- 파일 편집
- 셸 실행
- 패치 적용
- 웹 조회
- 작업 관리

MCP로 추가되는 전문 도구

- Android 디바이스 제어
- JADX 정적 분석
- Frida 런타임 후킹
- Ghidra 바이너리 분석
- Burp, ZAP, Chrome DevTools 연동

구분 원칙

- 기본 도구: Agent 자체의 작업 수행 능력
- MCP 도구: 외부 전문 시스템 접근 능력
- Skill: 두 계층을 감사 절차로 묶는 방식

9. 감사 기준은 MASVS와 MASTG에 정렬한다

왜 기준이 필요한가

- 도구 결과만으로는 Finding이 되지 않는다
- 영향도, 재현 가능성, 기준 매핑이 붙어야 산출물이 된다

도구 결과	기준 매핑 예시
하드코딩 비밀값	저장소, 암호화 통제
평문 HTTP 통신	네트워크 보안
Exported Component	플랫폼 상호작용
위험 WebView 설정	플랫폼/API 노출

목표 흐름

```
Tool Result -> Evidence -> MASVS Mapping -> Finding
```

10. Windows 기반 테스트 랩 구성

기본 구성 요소

- Windows 10 또는 11
- Python 3.10 이상
- Node.js LTS
- Git
- Android SDK Platform-Tools
- JDK
- Android 디바이스 또는 에뮬레이터
- MCP 지원 Client

운영 전 확인

```
python --version
node --version
git --version
adb version
adb devices
```

- 경로 표준화가 중요하다
- 버전과 장비 정보를 기록해야 한다
- 도구 루트를 정리하면 재설치와 팀 공유가 쉬워진다

11. MCP Client 설정 구조

설정 예시

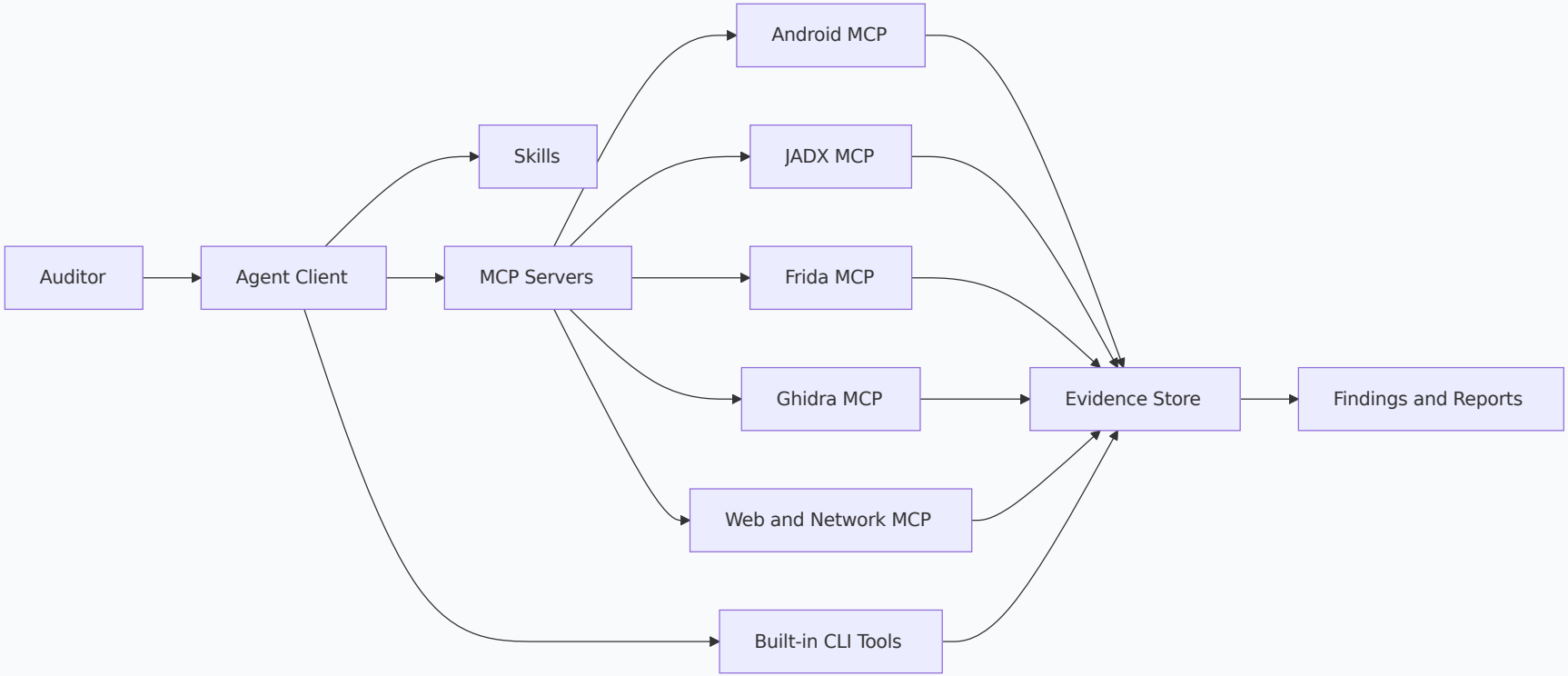
```
{
  "mcpServers": {
    "android": {
      "command": "uv",
      "args": ["--directory", "C:\\Tools\\android-mcp", "run", "server.py"]
    },
    "chrome-devtools": {
      "command": "npx",
      "args": ["-y", "chrome-devtools-mcp@latest"]
    }
  }
}
```

운영 팁

- Windows 경로 이스케이프 주의
- 서버별 로그와 버전 기록
- 위험 Tool 자동 승인 금지
- 설치 경로를 짧고 일관되게 잡으면 관리가 쉬움

12. 전체 감사 도구체인 아키텍처

전체 구조


Auditor
Agent Client
Skills
MCP Servers
Android MCP
JADX MCP
Frida MCP
Ghidra MCP
Web and Network MCP
Built-in CLI Tools
Evidence Store
Findings and Reports

핵심 원칙

- 모든 결과는 evidence 경로에 모은다
- 단계별로 필요한 서버만 켠다
- Agent는 조율자이지 최종 판단자가 아니다

13. 승인 범위와 Human-in-the-loop

승인 필요 작업

- 앱 설치와 삭제
- 파일 pull과 push
- Frida 스크립트 주입
- 능동 스캔
- 인증 우회 검증
- 운영 서비스 요청 재전송
- GitHub 이슈 생성과 수정

자동 실행 가능 작업

- 디바이스 정보 조회
- 화면 캡처
- 패키지 정보 확인
- 정적 결과 요약
- 로그 필터링
- 보고서 초안 생성

AI를 붙일수록 속도는 올라가지만, 실수도 빨라진다. 그래서 승인 게이트와 로그 보존이 필수다.

14. Android MCP의 역할

Android MCP가 다루는 것

- 디바이스 상태 확인
- 현재 화면 캡처
- UI Layout XML 수집
- logcat 수집
- 패키지와 권한 목록 확인
- 앱 실행 상태와 파일 수집

대표 Tool	용도
`get_device_info`	디바이스 기준선
`get_screenshot`	화면 증거
`get_uilayout`	UI 구조 분석
`get_logcat`	인증, 오류, 보안 이벤트
`get_package_info`	앱 메타데이터

15. Android MCP 적용 방법

```
adb devices
adb shell getprop ro.product.model
adb shell pm list packages
```

- ADB 연결 안정화가 먼저다
- 여러 디바이스가 있으면 serial 지정이 필요하다
- shell 실행은 allow-list 기반이 좋다
- 설치, 삭제, 파일 이동은 승인 후 수행한다

설정 포인트

- Platform-Tools 설치
- USB Debugging 활성화
- 테스트 디바이스 연결
- MCP Client 설정에 android 서버 등록

16. Android MCP 사용 예시

예시 요청

```
현재 실행 중인 앱 화면을 캡처해줘.
UI Layout에서 입력 필드와 버튼 목록을 정리하고,
Login 버튼의 중앙 좌표만 계산해줘.
```

기대 결과

- 스크린샷 저장
- XML에서 EditText와 Button 추출
- bounds 기반 좌표 계산
- 로그인 화면 증거를 이후 분석에 연결

이 단계에서 모은 화면과 UI 증거는 이후 인증 흐름 분석의 출발점이 된다.

17. JADX MCP의 역할

정적 분석에서 보는 것

- AndroidManifest
- 권한과 컴포넌트
- Exported 여부
- 하드코딩 Secret 후보
- 저장소와 인증 흐름
- WebView, 네트워크, 암호화 관련 코드 위치

감사 가치

- 수천 줄 코드를 빠르게 탐색
- Frida나 Ghidra 분석 대상을 선정
- Finding의 코드 위치 증거 확보

18. JADX MCP 적용 방법

적용 흐름

1. JADX 설치
2. JADX AI MCP 플러그인 설치
3. APK를 JADX에서 로드
4. MCP 서버 실행
5. Client에 서버 등록
6. 현재 클래스, 메서드, Manifest를 Agent가 읽도록 연결

주의점

- APK가 실제로 열려 있어야 한다
- 난독화된 코드 요약은 직접 검증해야 한다
- 민감 APK는 외부 전송 정책을 먼저 확인해야 한다

19. JADX MCP 사용 예시

예시 요청

```
현재 열려 있는 MainActivity를 요약해줘.
하드코딩된 API Key, Token, Password가 있는지 확인하고,
관련 코드 위치만 먼저 정리해줘.
```

기대 산출물

- Secret 후보 위치
- 저장소 관련 함수
- Exported Component 후보
- WebView 사용 클래스
- 후속 동적 검증 포인트

정적 분석 결과는 Finding 확정이 아니라, **사람이 더 깊게 볼 지점을 선별하는 재료**다.

20. Frida MCP의 역할

런타임에서 확인하는 것

- 함수 호출 인자와 반환값
- 복호화 후 실제 값
- 인증과 암호화 흐름
- 네트워크 전송 직전 데이터
- 동적 클래스 로딩과 네이티브 호출

감사 가치

- 정적 분석으로 안 보이는 실제 동작 확인
- 난독화된 로직의 행동 검증
- 인증과 토큰 처리 흐름 확인

21. Frida MCP 적용 방법

```
pip install frida-tools
frida-ps -U
```

```
adb push frida-server /data/local/tmp/frida-server
adb shell "chmod 755 /data/local/tmp/frida-server"
adb shell "/data/local/tmp/frida-server &"
frida-ps -U
```

적용 시 주의점

- Frida와 frida-server 버전 일치
- spawn 방식이 필요한 후킹 구분
- 테스트 계정만 사용
- 함수명과 클래스명을 먼저 검토

22. Frida MCP 사용 예시

예시 요청

```
LoginManager의 validateUser 메서드를 추적해줘.
호출 인자와 반환값은 마스킹해서 로그로 남기고,
성공과 실패 조건만 짧게 요약해줘.
```

기대 결과

- 함수 호출 로그
- 입력값 형태와 마스킹 정보
- 성공과 실패 분기 요약
- 정적 분석 코드 위치와 연결

23. Ghidra MCP의 역할

네이티브 분석에서 확인하는 것

- JNI 함수
- C, C++ 네이티브 코드
- 암호화와 복호화 루틴
- 문자열 복호화 로직
- 안티디버깅과 안티탬퍼링
- import, export, cross-reference

실무적 가치

- Java 코드 바깥의 로직을 확인할 수 있다
- Frida 후킹 대상을 고르는 데 도움된다
- 클라이언트 안에 숨겨진 보안 로직을 드러낸다

24. Ghidra MCP 적용 방법

적용 흐름

1. Ghidra 설치
2. JDK 준비
3. APK에서 `.so` 추출
4. Ghidra 프로젝트에 로드
5. Auto Analysis 실행
6. Ghidra MCP 플러그인과 Bridge 연결
7. Client에서 서버 등록

주의점

- 파일이 실제로 열려 있어야 한다
- 포트 충돌 확인
- 자동 리네이밍은 제안으로만 쓰고 사람이 검토한다

25. WebView와 네트워크 분석

Chrome DevTools, Playwright, Burp 계열 도구

- DOM 구조 분석
- 로그인 플로우 자동화
- Network 요청과 응답 확인
- LocalStorage, Cookie, SessionStorage 확인
- 콘솔 에러와 CSP 점검

주의점

- 페이지 내용에 프롬프트 인젝션이 포함될 수 있다
- 민감 토큰과 쿠키는 마스킹한다
- 능동 스캔과 요청 재전송은 승인 후 수행한다

26. 자동 스캐닝 스택

MobSF, Semgrep, CodeQL의 위치

도구	역할	해석 원칙
MobSF	APK 초벌 정적 분석	결과를 그대로 확정하지 않음
Semgrep	반복 가능한 룰 기반 탐지	오탐과 중복 제거 필요
CodeQL	데이터플로우 중심 질의	추가 검증과 코드 맥락 필요

한 줄 원칙

자동 스캐닝은 Finding 생성기가 아니라 **검증 후보를 빠르게 모으는 도구**다.

27. AI Mobile Audit Orchestrator

단계별 실행 흐름

Scope
Baseline
Static analysis
Dynamic analysis
Native analysis
Web and network
Evidence normalization
MASVS mapping
Finding draft

목표

- 도구 결과를 하나의 감사 컨텍스트로 묶는다
- 반복 수집 작업을 표준화한다
- MASVS 기준의 Finding 초안을 만든다
- 사람의 최종 판단을 남긴다

28. Finding 생성 템플릿

```
# Finding: 제목

## 요약
## 영향도
## 영향 범위
## 재현 절차
## 증거
## 기준 매핑
## 권고 조치
## 검증 방법
```

AI와 사람의 역할 분담

- AI가 돕는 것
  - 초안 작성
  - 증거 연결
  - 중복 병합
  - 권고안 문장화
- 사람이 확정하는 것
  - 실제 영향도
  - 재현 가능성
  - 위험도 확정
  - 고객 맥락 반영

29. Skill 설계

Skill에 들어가야 할 것

- 언제 이 Skill을 쓸지
- 입력값 목록
- 사용 가능한 Tool 목록
- 단계별 절차
- 금지 작업과 승인 필요 작업
- 보고서 형식과 템플릿

단계별 Tool 매핑이 중요하다

- 대상 식별: Android MCP
- Manifest 분석: JADX, MobSF
- Secret 탐지: Semgrep, JADX
- 인증 분석: JADX, Frida, Burp
- 네트워크 분석: Burp, Chrome DevTools
- 네이티브 분석: Ghidra, Frida
- 보고서화: Skill Template

30. Evidence Bundle 표준화

```
evidence/{app}/{date}/
  00_scope/
  01_device/
  02_static/
  03_dynamic/
  04_native/
  05_network/
  06_findings/
```

메타데이터에 남길 것

- 대상 앱명과 버전
- 테스트 일시
- 디바이스 정보
- 도구 버전
- 실행 명령 또는 Tool Call 요약
- 증거 파일 경로
- 마스킹 여부

증거 구조가 표준화되어야 보고서 검토와 재검증이 쉬워진다.

31. 리스크와 운영 통제

주요 리스크

- 과도한 Tool 권한
- 셸 명령 오남용
- 민감정보 유입
- 프롬프트 인젝션
- 검증되지 않은 MCP Server
- 승인 없는 자동 실행
- 범위 초과 스캔

대응 원칙

- 최소 권한 토큰
- allow-list 기반 대상 제한
- 위험 작업 승인 게이트
- secret redaction 자동화
- Tool Call 로그 보존
- 테스트 환경 격리
- Human-in-the-loop 유지

32. 결론

최종 정리

- MCP는 연결 규격이다
- MCP Server는 Tool 묶음이다
- Tool은 실행 단위다
- Skill은 감사 절차와 품질 기준이다
- Agent는 여러 도구를 조율하는 오케스트레이터다
- 판단과 책임은 감사자에게 남아 있어야 한다

Q&A

- 설치와 운영 이슈
- 적절한 도구 조합
- Skill 표준화 방법
- AI 초안 검증 절차

33. 참고 자료

핵심 참고 링크

- Model Context Protocol Specification
- MCP Basic Concepts
- Claude Code Tools Reference
- Claude Code MCP
- Claude Code Skills
- OpenAI Codex MCP
- OpenAI Codex Config Reference
- OWASP MASVS
- OWASP MASTG

도구별 링크

- Android MCP Server
- JADX MCP
- JADX AI MCP
- Frida MCP
- GhidraMCP
- Chrome DevTools MCP
- Playwright MCP
- Semgrep MCP
- CodeQL Development MCP Server
- MobSF
- GitHub MCP Server